

Lecture 7

Attention Mechanism

P1. 注意力提示

{ 非自主性提示 只考虑非自主线索 $\rightarrow$ 卷积、全连接、池化层

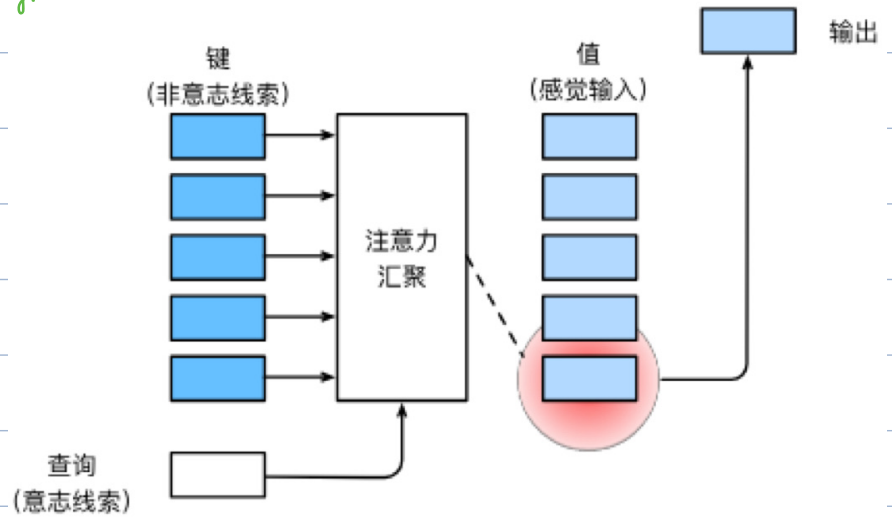
{ 自主性提示 考虑自主线索 $\rightarrow$ 注意力机制

自主性提示 $\rightarrow$ 查询 Query. ^{独立线索}

注意力汇聚层

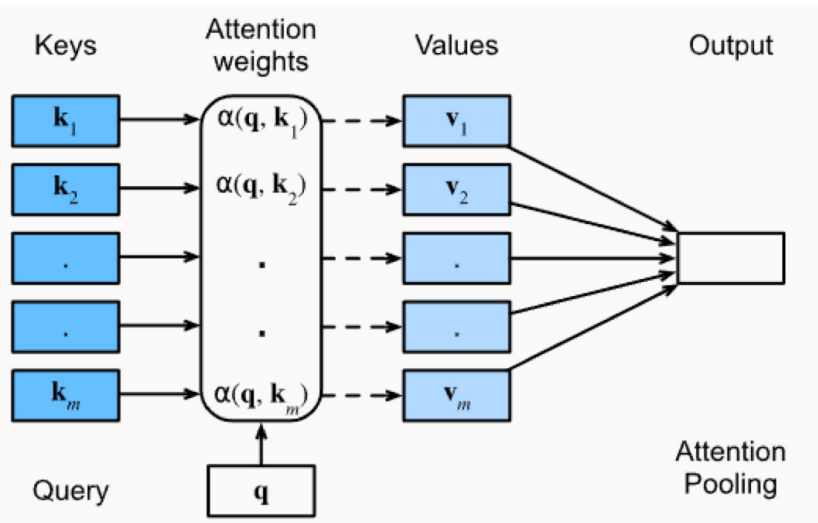
$\downarrow$ Attention pooling.
有偏向性选择某种输入

输入 $\rightarrow$ 值 value 与 非自主线索 key 取对 ^{独立输入} ^{键, 非独立线索}



P2. 注意力汇聚

参数注意力汇聚: $f(x) = \sum_i \alpha(x, x_i) y_i$ ^{用 Query 和 Key 的相似性作为权重}



非参数注意力汇聚: ① 平均汇聚 $f(x) = \frac{1}{n} \sum_{i=1}^n y_i$

② 核回归 $f(x) = \frac{\sum_{i=1}^n \frac{K(x-x_i)}{\sum_{j=1}^n K(x-x_j)} y_i$

k 取高斯核 $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-\frac{u^2}{2})$

$$f(x) = \sum_{i=1}^n \text{softmax}(-\frac{1}{2}(x-x_i)^2) y_i$$

$$\text{引入可学习 } w \Rightarrow f(x) = \sum_{i=1}^n \text{softmax}(-\frac{1}{2}((x-x_i)w)^2) y_i$$

$$\text{写作 } f(x) = \sum_i \alpha(x, x_i) y_i$$

P3. 注意力函数

$$f(x) = \sum_i \alpha(x, x_i) y_i = \sum_{i=1}^n \text{softmax}(-\frac{1}{2}[(x-x_i)w]^2) y_i$$

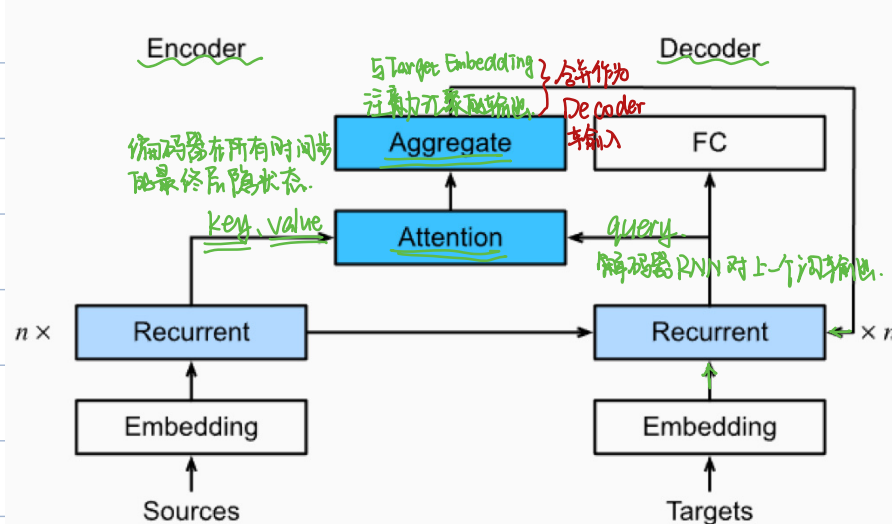
注意力函数 注意力权重 注意力分数 权重 q, k 相似度
权重是分数经 softmax 结果

q, k 经单输入单输出隐藏层 MLP

算法: ① 加性注意力 $A(q, k) = w_v \tanh(W_q q + W_k k)$
权重

② 缩放点积注意力 Key 和 query 长度相同
 q, k 内积. 点积. $A(q, k) = qk^T / \sqrt{d_k}$

P4. 使用注意力机制的 seq2seq 机器翻译中, 每个生成词可能相关于源句子中不同词 seq2seq 不直接建模



seq2seq 通过隐状态在编码器和解码器中传递信息, RNN 最后隐藏层加入 Decoder 输入

Attention Mechanism 根据解码器 RNN 输出匹配合适编码器 RNN 输出

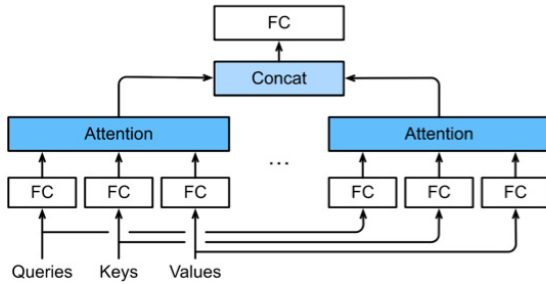
更有效传递信息

RNN 所有隐藏信息加入 Decoder 输入 根据 AM 加权

Transformer

P1. Multi-Head Attention 多头注意力

- 对同一key, value, query, 希望抽取不同的信息
 - 例如短距离关系和长距离关系
- 多头注意力使用 h 个独立的注意力汇聚
 - 合并各个头(head)输出得到最终输出



P2. Self Attention and Position Encoding 输入中加入位置信息 使自注意力记忆位置信息

Key, Value, Query 来自同样的数据

SA 无位置信息记录 → 位置编码

自编码输入: $x + p_{\text{position}}$

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right) \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

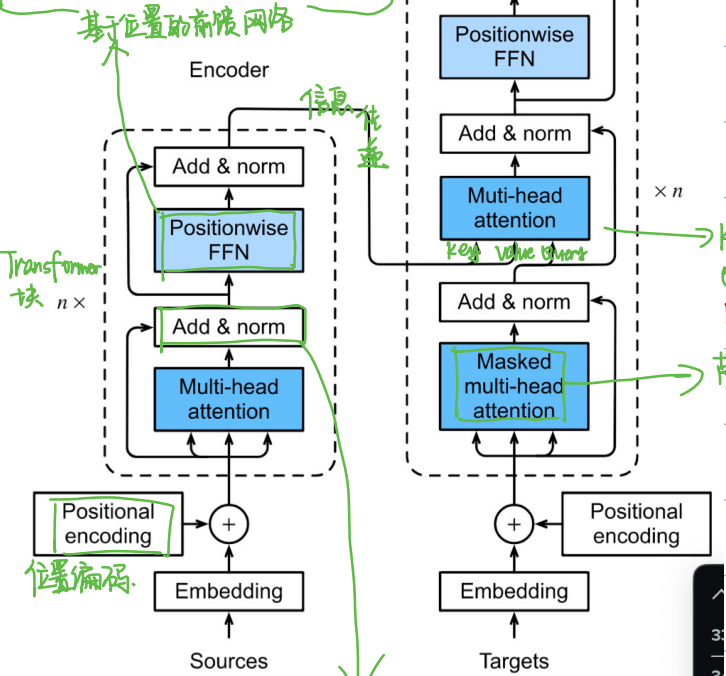
$\downarrow$ 偶数索引
第 j 个样本

变化 $\sin/\cos$ 函数频率变化 一旦位置确定 位置编码也随之确定

P3. Transformer

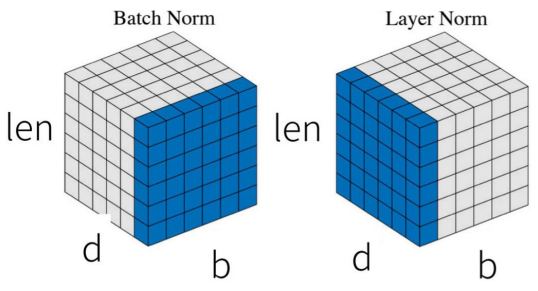
去掉 RNN, 纯注意力机制的编码、解码器

- 将输入形状由 (b, n, d) 变换成 (bn, d)
- 作用两个全连接层
- 输出形状由 (bn, d) 变换回 (b, n, d)
- 等价于两层核窗口为1的一维卷积层



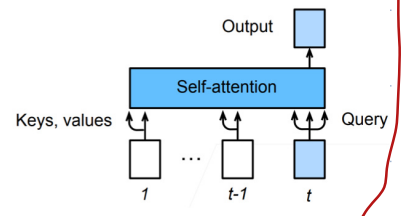
Key, value 来自 Encoder 输出
Query 来自目标序列
Decoder 和 Encoder 中 Block 个数和输出维度一样。
带掩码的多头注意力
解码器对序列中一个元素输出时，不应该考虑该元素之前的元素
掩码实现：计算 x_i 输出时假设当前序列长度为 i

- 批量归一化对每个特征/通道里元素进行归一化
 - 不适合序列长度会变的 NLP 应用
- 层归一化对每个样本里的元素进行归一化



预测

- 预测第 $t+1$ 个输出时
- 解码器中输出前 t 个预测值
 - 在自注意力中，前 t 个预测值作为 key 和 value，第 t 个预测值还作为 query



Tranformer 去掉RNN 纯自注意力机制。

块: Encoder 多头自注意力 线性连接 前馈网络 ^{线性组合多头}
(位置编码)

Normalization
Add + Norm layer.

Decoder 来自目标序列 } 交叉注意力
K, V 来自 Encoder 输出.
解码 - 交叉注意力
Add & Norm
FFN 同样 n 个 Tranformer 块.

解码器只看前边的 mask 掩码

全连接层 等价于两层核大小为 1 的卷积层
输入 $(b, n, d) \rightarrow (b, n, d)$
输出 $(b, n, d) \rightarrow (b, n, d)$

层归一化 Batchnorm 不适合对每个特征 / channel 归一化
对每个样本元素归一化。

FFN 线性组合
前馈网络